



Rozpoznawanie pytań na podstawie analizy głosu

Kacper Jurek, Mateusz Łopaciński

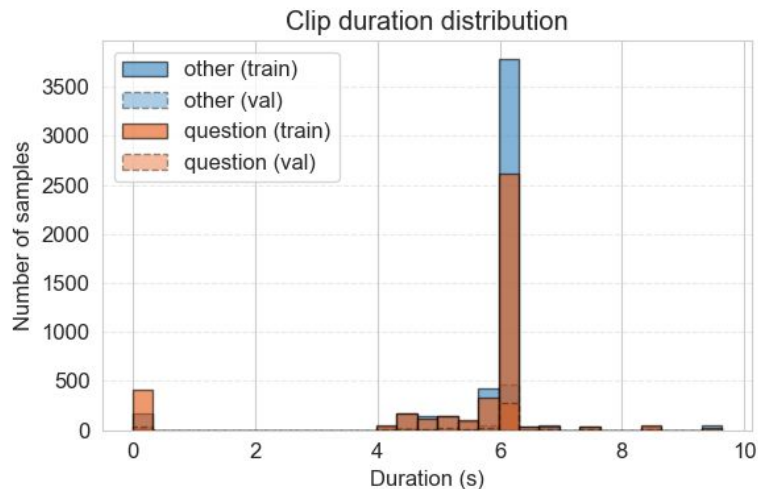
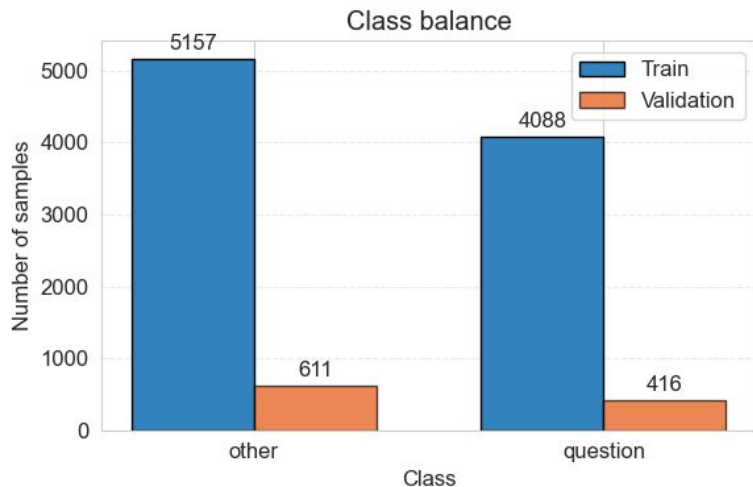


Przypomnienie problemu

- **Cel:** automatycznie rozpoznać, czy wypowiedź w nagraniu jest pytaniem, czy twierdzeniem.
- **Intonacja:** pytania często kończą się charakterystycznym, wznoszącym konturem wysokości tonu.
- **Wyzwania:** duża różnorodność głosów, akcentów i warunków nagrania wymaga modelu odpornego na zmienność.

Wstępna analiza danych

W obu podzbiorach zdecydowanie dominuje klasa „other”, a rozkład długości klipów wskazuje, że typowe nagranie trwa około 6 s, zaledwie kilkoma istotnie krótszymi lub dłuższymi przypadkami.





Fine-tuning Hiper Parametrów

- **Wykorzystano:** Optuna 3.x, MedianPruner (wcześnie ucina nie-rokujące próby)
- **Cel:** zautomatyzowana optymalizacja architektury sieci i parametrów uczenia
- **Badane zmienne:**
 - liczba kanałów w warstwach konwolucyjnych conv1, conv2
 - rozmiar i liczba warstw części rekurencyjnej lstm_hidden, lstm_layers
 - współczynnik dropout
 - współczynniki uczenia lr oraz weight_decay



```
# Najlepsze hiperparametry
best_params = {
    "conv1": 48,
    "conv2": 96,
    "lstm_hidden": 128,
    "lstm_layers": 2,
    "dropout": 0.10,
    "lr": 0.001079348466540913,
    "weight_decay": 0.00010567170444977611,
}
```

Architektura

- **CNN:** warstwy konwolucyjne wydobywają lokalne wzorce z mel-spektrogramu.
- **Bi-LSTM:** dwukierunkowy LSTM uczy się kontekstu czasowego.
- **Lekka sieć:** 3.72 mln parametrów

```
class CRNN(nn.Module):
    def __init__(self, conv1=48, conv2=96, lstm_hidden=128, lstm_layers=2, dropout=0.1):
        super().__init__()

        self.cnn = nn.Sequential(
            nn.Conv2d(1, conv1, 3, padding=1),
            nn.BatchNorm2d(conv1),
            nn.ReLU(),
            nn.MaxPool2d(2),
            nn.Conv2d(conv1, conv2, 3, padding=1),
            nn.BatchNorm2d(conv2),
            nn.ReLU(),
            nn.MaxPool2d(2),
        )

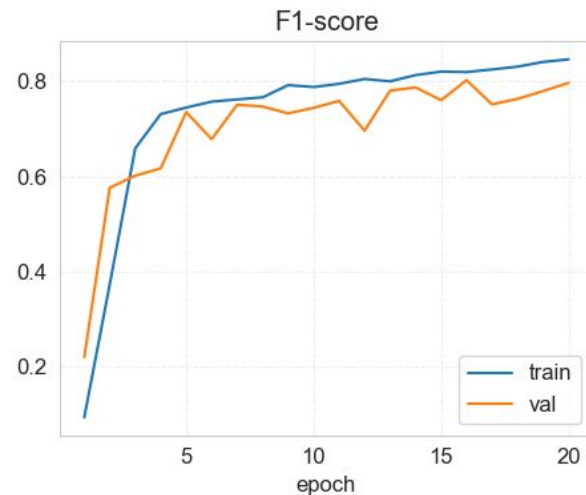
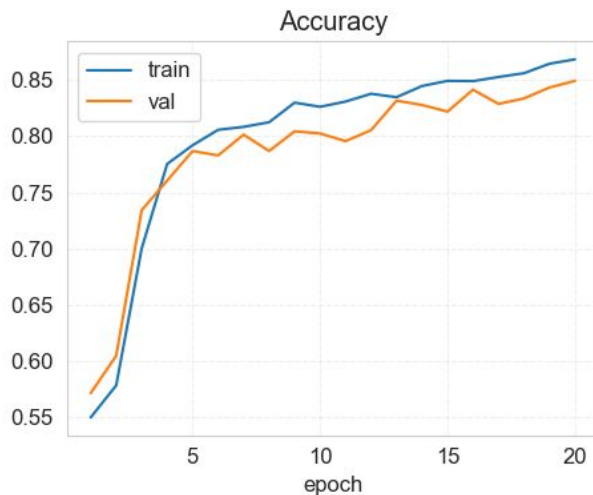
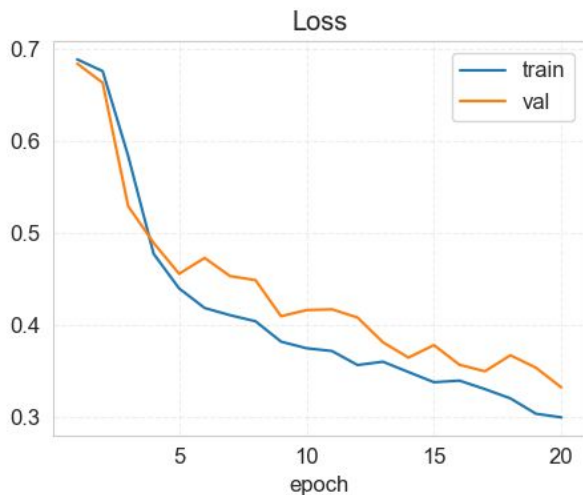
        with torch.no_grad():
            dummy = torch.zeros(1, 1, N_MELS, 100)
            c, f, t = self.cnn(dummy).shape[1:]

        self.lstm = nn.LSTM(
            input_size = conv2 * f,
            hidden_size = lstm_hidden,
            num_layers = lstm_layers,
            batch_first = True,
            bidirectional= True,
            dropout = dropout if lstm_layers > 1 else 0.0
        )
        self.fc = nn.Linear(lstm_hidden * 2, 2)

    def forward(self, x):
        x = self.cnn(x.unsqueeze(1))
        b, c, f, t = x.shape
        x = x.reshape(b, c * f, t).permute(0, 2, 1)
        x, _ = self.lstm(x)
        return self.fc(x[:, -1])
```

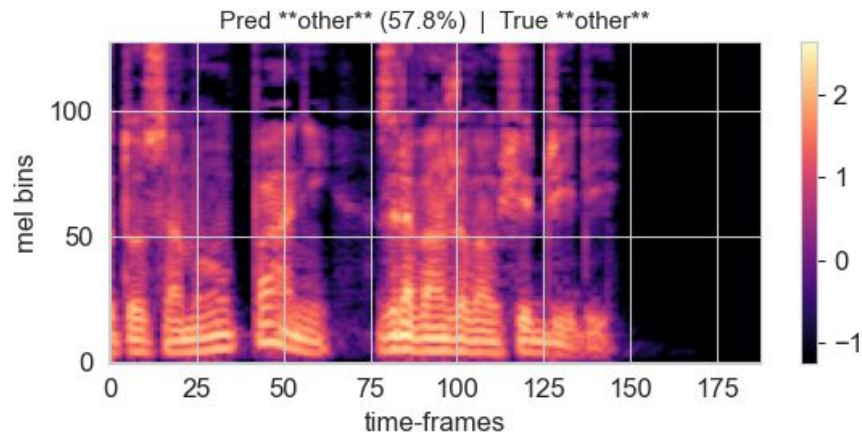
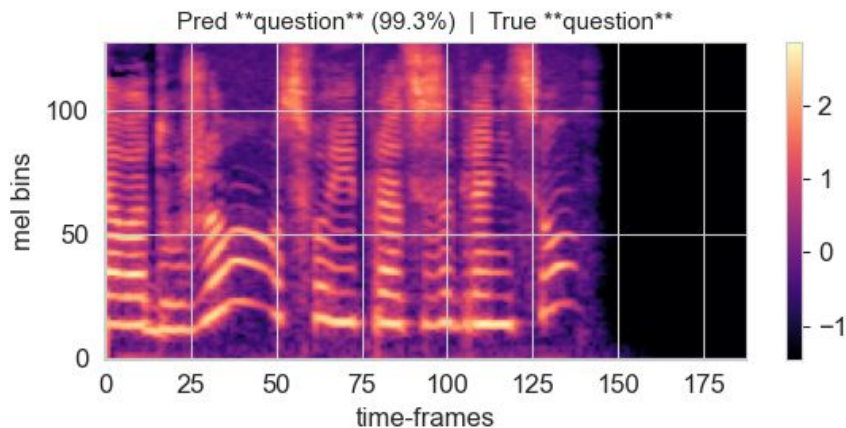
Postęp uczenia

Krzywe straty maleją, a Accuracy i F1 systematycznie rosną w obu zbiorach, co wskazuje, że model uczy się skutecznie i zachowuje dobrą ogólną zdolność generalizacji bez przeuczenia.



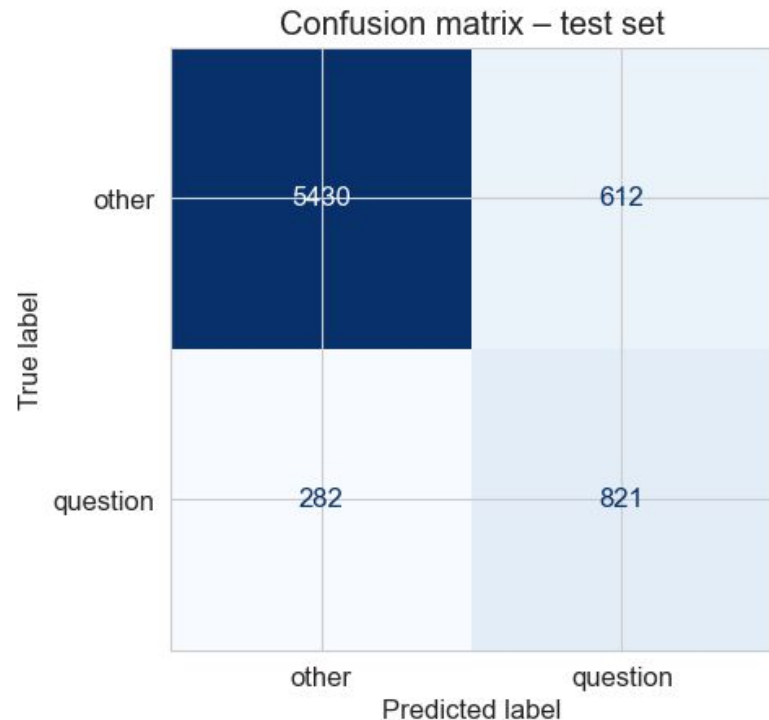
Przykładowe spektrogramy MEL i predykcje modelu

- Model poprawnie klasyfikuje większość próbek dźwięku z wysoką pewnością.



Macierz pomyłek

- Zbiór testowy jest wyraźnie nie zrównoważony ($6042 \times$ „other” vs $1103 \times$ „question”).
- Dlatego liczbowo widzimy więcej błędów przy klasie other (612) – to efekt dużej przewagi jej próbek.
- Procentowo pomyłki to ok. 10% dla other i 26% dla question – pytania pozostają trudniejsze.





Podsumowanie

- Rekurencyjna sieć **CNN + Bi-LSTM** dobrze radzi sobie z rozróżnianiem pytań i stwierdzeń.
- Większość pytań jest wychwytywana; około 1/4 wciąż jednak błędnie trafia do klasy „stwierdzenie”.
- Taki profil błędów jest jednak lepszy niż omylne klasyfikowanie stwierdzeń jako pytania
- Automatyczne strojenie hiperparametrów (Optuna) pozwoliło na osiągnięcie lepszych wyników niż wyniki osiągnięte przez wcześniej wykorzystywany model z ręcznie dobranymi hiperparametrami



Dziękujemy za uwagę!

Kacper Jurek, Mateusz Łopaciński